

**Notebook Link:** The shareable link to the notebook is provided at the top, ensuring accessibility for anyone with the link, as required.

## **CS 412: Machine Learning Homework 1 Report**

### **Title: Breast Cancer Classification Using k-Nearest Neighbors and Decision Tree Models**

**Name:** Abdallah Al Homsı

**Student ID:** 34565

**Submission Date:** 9 March 2026

**Semester:** Spring 2026

## Table Of Contents

|                                           |          |
|-------------------------------------------|----------|
| <b>1. Methodology.....</b>                | <b>3</b> |
| 1.1 Dataset.....                          | 3        |
| 1.2 Data Splitting.....                   | 3        |
| 1.3 Data Analysis.....                    | 4        |
| 1.4 Data Preprocessing.....               | 4        |
| <b>2. Results.....</b>                    | <b>4</b> |
| 2.1 k-NN Classifier.....                  | 4        |
| 2.2 Decision Tree Classifier.....         | 4        |
| <b>3. Misclassification Analysis.....</b> | <b>4</b> |
| <b>4. Conclusion.....</b>                 | <b>4</b> |

## 1. Methodology

### 1.1 Dataset

The Breast Cancer Wisconsin Dataset was worked on for this homework. This specific dataset has 569 samples with 30 numerical features being extracted from the images. These features shape the characteristics and descriptions of the nuclei like radius, texture, perimeter, area, and concavity.

The goal is a binary classification problems in which every sample is labeled as 2 different classes:

- *Malignant (Cancerous)*
- *Benign (Non Cancerous)*

As the dataset was loaded and imported using the `load_breast_cancer()` function assisted by the *scikit-learn library*.

```
1 # Import Libraries and Dataset, and Loading of Dataset
2
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from sklearn.model_selection import train_test_split
6
7 from sklearn.datasets import load_breast_cancer
8 data = load_breast_cancer()
9 X,y = data.data, data.target
10
11 # Print to Ensure Loading of Data is Successful
12
13 print("Dataset shape:", X.shape)
14 print("Number of features:", X.shape[1])
15 print("Feature names:", data.feature_names)
16 print("Target names:", data.target_names)
17
18
19
```

```
... Dataset shape: (569, 30)
Number of features: 30
Feature names: ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
'mean smoothness' 'mean compactness' 'mean concavity'
'mean concave points' 'mean symmetry' 'mean fractal dimension'
'radius error' 'texture error' 'perimeter error' 'area error'
'smoothness error' 'compactness error' 'concavity error'
'concave points error' 'symmetry error' 'fractal dimension error'
'worst radius' 'worst texture' 'worst perimeter' 'worst area'
'worst smoothness' 'worst compactness' 'worst concavity'
'worst concave points' 'worst symmetry' 'worst fractal dimension']
Target names: ['malignant' 'benign']
```

## 1.2 Data Splitting

The Breast Cancer Wisconsin Dataset was split into three sets:

| Dataset        | Percentage | Purpose                        |
|----------------|------------|--------------------------------|
| Training Set   | 70%        | To Train The Models            |
| Validation Set | 15%        | Used for Hyperparameter Tuning |
| Test Set       | 15%        | Used for Final Evaluation      |

The split was performed using `train_test_split()` with `random_state = 42` to ensure reproducibility.

```
5 X_train, X_temp, y_train, y_temp = train_test_split(
6     X, y,
7     test_size=0.30,
8     random_state=42
9 )
10
11 X_val, X_test, y_val, y_test = train_test_split(
12     X_temp, y_temp,
13     test_size=0.50,
14     random_state=42
15 )
16
17 # Printing of Training, Validation, and Test shapes
18
19 print("Training set:", X_train.shape)
20 print("Validation set:", X_val.shape)
21 print("Test set:", X_test.shape)
```

```
... Training set: (398, 30)
Validation set: (85, 30)
Test set: (86, 30)
```

## 1.3 Data Analysis

Before preprocessing, analysis of the data took place in order to know the structure of the dataset.

The class distribution was examined to know if the dataset was balanced or skewed

The dataset displays that it is not extremely skewed, however it is a bit unbalanced and skewed towards the Benign tumors, as we can see in the code above that **Malignant Percentage is 37% and Benign is approximately 63%.**

```

1 # Count number of malignant and benign samples
2
3 malignant_count = np.sum(y == 0)
4 benign_count = np.sum(y == 1)
5
6 print("Class distribution:")
7 print("Malignant (0):", malignant_count)
8 print("Benign (1):", benign_count)
9
10 total = len(y)
11 print("\nTotal samples:", total)
12 print("Malignant percentage:", malignant_count/total)
13 print("Benign percentage:", benign_count/total)

```

```

... Class distribution:
Malignant (0): 212
Benign (1): 357

Total samples: 569
Malignant percentage: 0.37258347978910367
Benign percentage: 0.6274165202108963

```

Not only that, part of the analysis were the computation of the basic statistics like (mean and standard deviation) for the first three features:

- *Mean Radius*
- *Mean Texture*
- *Mean Perimeter*

These basic statistics provide the scale of the features and the distribution within the data.

```

1 # Printing the Mean and the Std of the first 3 features of the dataset
2
3 for i in range(3):
4
5     mean_value = np.mean(X[:, i])
6     std_value = np.std(X[:, i])
7
8     print(data.feature_names[i])
9     print("Mean:", mean_value)
10    print("Standard deviation:", std_value)
11    print()

```

```

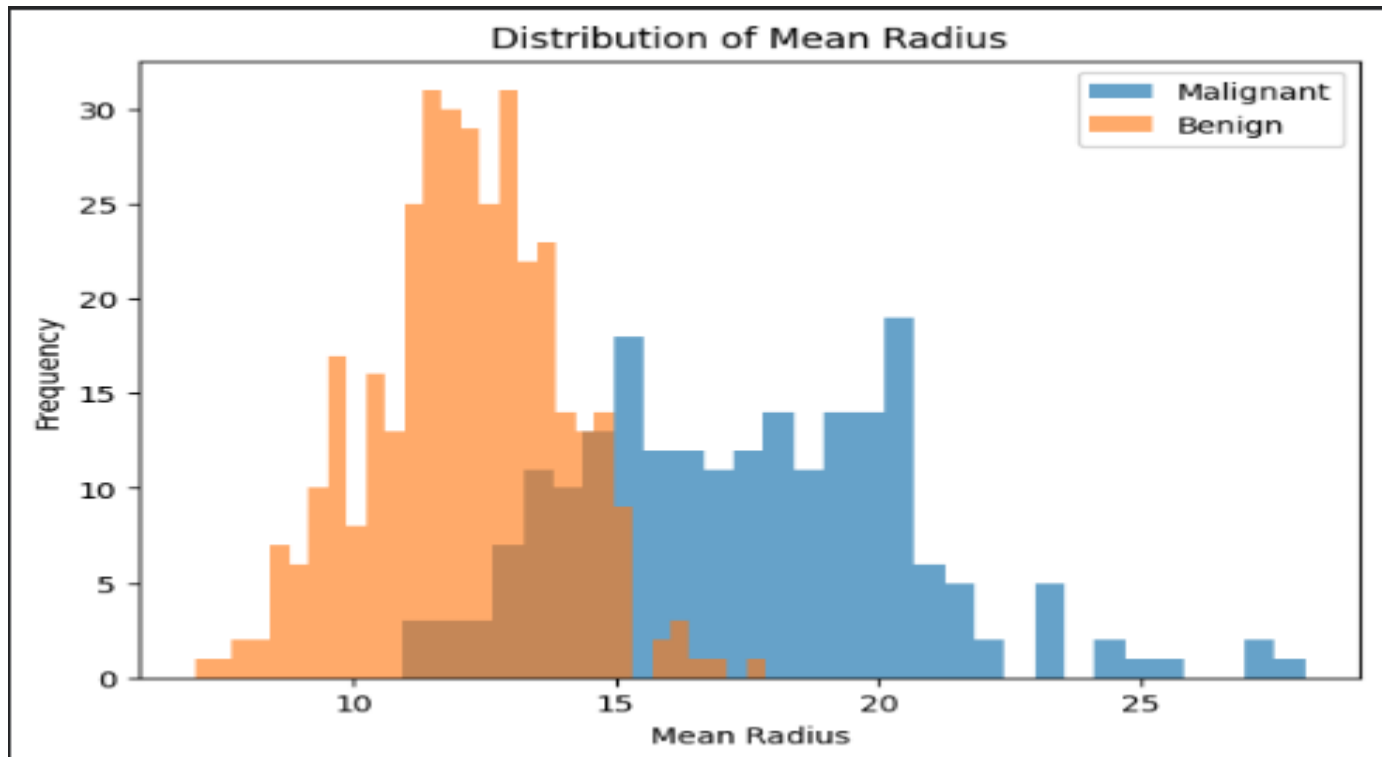
... mean radius
Mean: 14.127291739894552
Standard deviation: 3.520950760711062

mean texture
Mean: 19.289648506151142
Standard deviation: 4.297254637090421

mean perimeter
Mean: 91.96903339191564
Standard deviation: 24.27761929305318

```

Also, the histogram of the *mean radius* was plotted with the help of *matplotlib library* to show the distribution of it between the two classes.



**Interpretation:** The histogram shows the distribution of the *mean radius* feature for the two tumor classes. In general, malignant tumors tend to have larger radius values compared to benign tumors, indicating that tumor size may be an informative characteristic for classification.

## 1.4 Data Preprocessing

Before moving onto the training, we need to standardize the dataset so that each single feature has:

- *Mean = 0*
- *Standard Deviation = 1*

This step emphasizes a very important step for especially one of the algorithms we are working with, the k-NN algorithm since it relies on distance calculation between samples.

**The reason:** Algorithms that rely on calculations like the k-Nearest Neighbor algorithm are extremely sensitive to the scale of the input features, meaning that if features have distinct ranges those with larger numerical value will always dominate the other features making it a bias model.

These standardized parameters must be computed only on the training dataset to avoid data leakage. This occurs when the information from the validation or test datasets are being used in training which cancels out the generalization process of having the actual accuracy of the model, since the model will train on data that must be unseen. So we apply it on the training dataset only so the model would behave fairly and reduce the risk of misleading results resembling overfitting.

```
1 mean_train = np.mean(X_train, axis=0)
2 std_train = np.std(X_train, axis=0)
3
4 X_train_std = (X_train - mean_train) / std_train
5 X_val_std = (X_val - mean_train) / std_train
6 X_test_std = (X_test - mean_train) / std_train
```

## 2. Results

### 2.1 k-NN Classifier

The **k-NN Algorithm** is a supervised learning method used in classification tasks. The algorithm predicts the class label based on the labels of k closest samples of the new sample that is being predicted to which class it belongs to, meaning the new sample belongs to the class determined by majority voting among the k nearest neighbors.

Due to the reliance on distance calculations, the standardization is a vital step which we did in the preprocessing step to make sure all the features are contributing equally to the distance computation.

The key hyperparameter is the K value:

- **Small Value of K:** leads to overfitting
- **Large Value of K:** leads to underfitting

To determine the most optimal K value for the model multiple values were taken to be tested upon

$k \in \{1, 3, 5, 7, 9, 15, 21\}$

First of all k-NN classification needs were imported

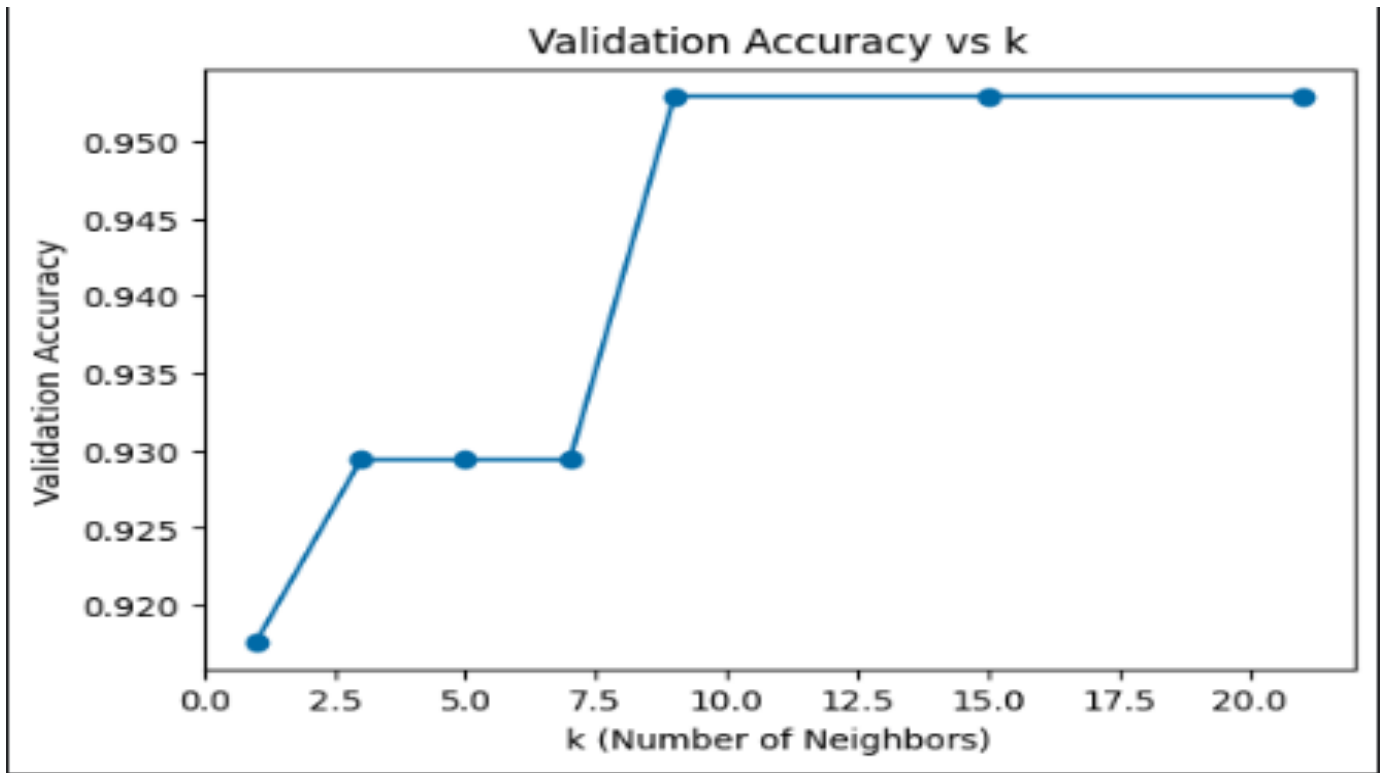
```
1 # Import the classifier
2
3 from sklearn.neighbors import KNeighborsClassifier
4 from sklearn.metrics import accuracy_score
```

After that, for each value of k, the model was trained using the training set we split earlier and then evaluated on the validation set. The validation accuracy was taken for each configuration to compare their performances

```
1 #Checking Every K Value
2
3 k_values = [1,3,5,7,9,15,21]
4
5 validation_accuracies = []
6
7 for k in k_values:
8
9     model = KNeighborsClassifier(n_neighbors=k)
10
11     model.fit(X_train_std, y_train)
12
13     y_val_pred = model.predict(X_val_std)
14
15     acc = accuracy_score(y_val, y_val_pred)
16
17     validation_accuracies.append(acc)
18
19     print("k =", k, "Validation Accuracy =", acc)
```

```
k = 1 Validation Accuracy = 0.9176470588235294
k = 3 Validation Accuracy = 0.9294117647058824
k = 5 Validation Accuracy = 0.9294117647058824
k = 7 Validation Accuracy = 0.9294117647058824
k = 9 Validation Accuracy = 0.9529411764705882
k = 15 Validation Accuracy = 0.9529411764705882
k = 21 Validation Accuracy = 0.9529411764705882
```





The plot illustrates how validation accuracy changes as the number of neighbors increases. Initially, performance improves as  $k$  increases because the model becomes less sensitive to noise in the training data. However, after a certain point the improvement stabilizes.

In this experiment, the highest validation accuracy was obtained for  $k = 9, 15, \text{ and } 21$ . When multiple values produce the same validation accuracy, the smallest value is typically preferred because it results in a simpler model with a more flexible decision boundary. Therefore,  $k = 9$  was selected as the optimal number of neighbors.

After selecting the best value of  $k$ , the final  $k$ -NN model was retrained using both the training and validation sets combined. The final model was then evaluated on the test set to measure its performance on unseen data.

The trained model was evaluated on the test set, which contains the data that was not used during training so it provides an unbiased estimate of how well the model performed.

The final  $k$ -NN model achieved a test accuracy of **98.8%**, showing that the model is highly effective at distinguishing between the malignant and benign tumors in the dataset.

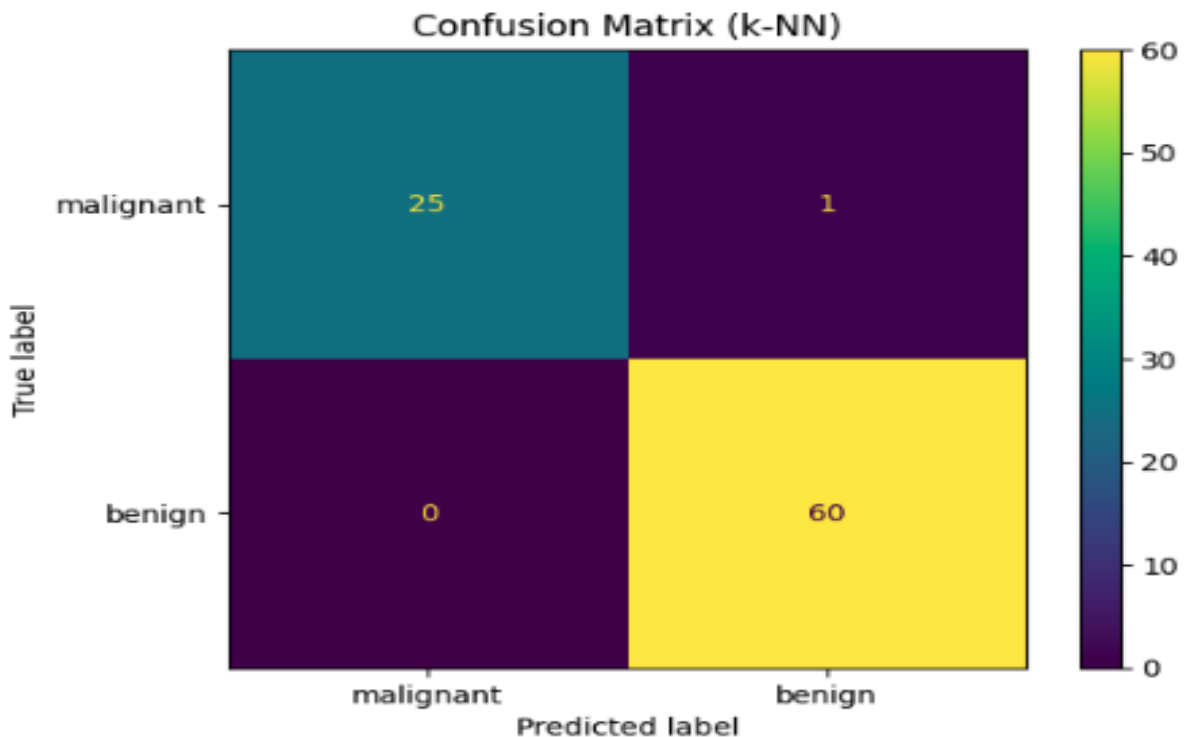
The figure below shows how the final model performed and the way it was predicting based on the test dataset.

```
1 X_train_val = np.vstack((X_train_std, X_val_std))  
2 y_train_val = np.concatenate((y_train, y_val))
```

```
1 final_knn = KNeighborsClassifier(n_neighbors=best_k)  
2  
3 final_knn.fit(X_train_val, y_train_val)  
4  
5 y_test_pred = final_knn.predict(X_test_std)
```

```
1 test_accuracy = accuracy_score(y_test, y_test_pred)  
2  
3 print("Test Accuracy:", test_accuracy)
```

Test Accuracy: 0.9883720930232558



The Confusion Matrix displays a detailed view of the performance of the final model:

- 25 malignant tumors were correctly classified as malignant (True Positives).
- 1 malignant tumor was incorrectly classified as benign (False Negative).
- 60 benign tumors were correctly classified as benign (True Negatives).
- 0 benign tumors were incorrectly classified as malignant (False Positives).

This shows only a single misclassification on the dataset, however predicting a malignant tumor as benign is a critical error. However, the model makes only one such error.

## 2.2 Decision Tree Classifier

The **Decision Tree Algorithm** is a supervised learning method used for classification tasks. Not like k-NNs, this algorithm classifies data by recursively splitting the dataset based on the feature values. As it separates based on the best at the current level of the tree making it a Greedy algorithm.

Two important hyperparameters were tuned to control the complexity of the model:

- **Max\_depth:** the maximum depth of the tree
- **Min\_samples\_split:** the minimum number of samples required to split the node

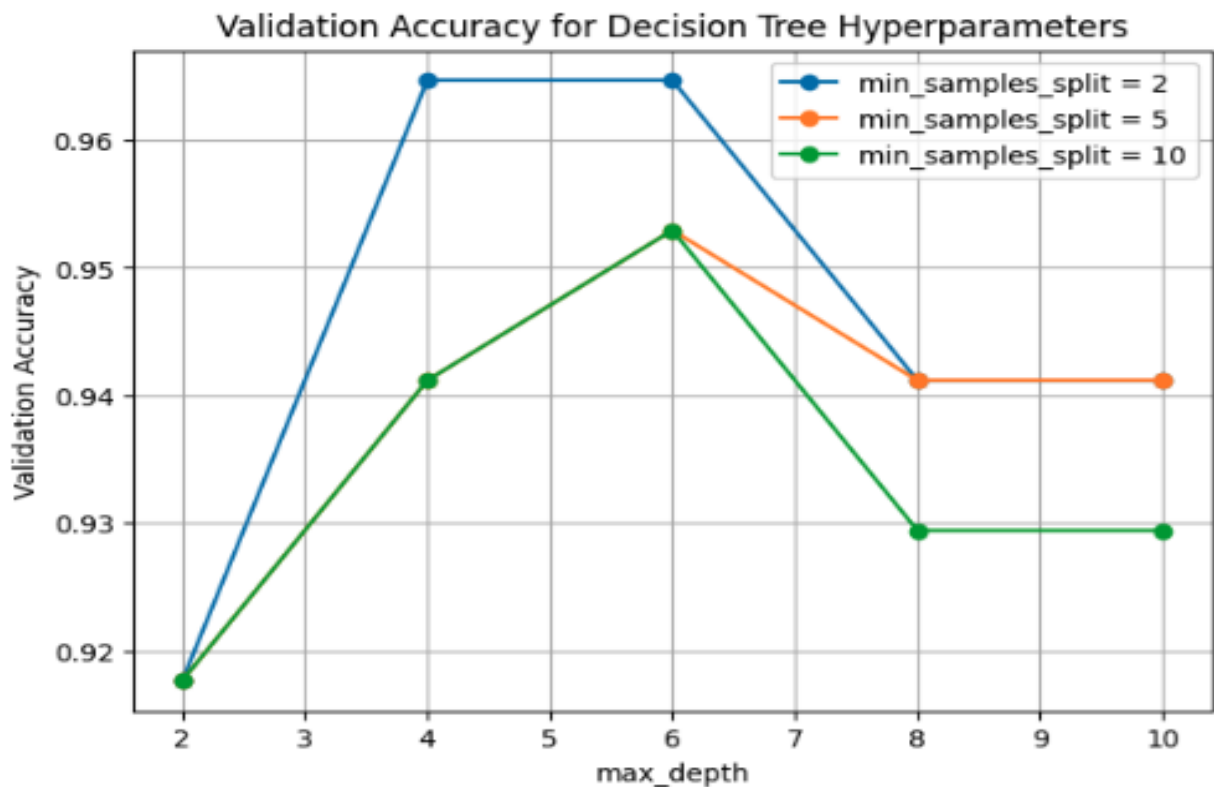
Testing on different parameters in order to avoid the model becoming too complex leading to overfitting.

The following values were tested on:

```
max_depth_values = [2, 4, 6, 8, None]
min_samples_split_values = [2, 5, 10]
```

For each combination (nested loop), the model was trained using the training set and evaluated on the validation set. Recording the highest validation accuracy of a certain combination to mark as the best hyperparameters.

```
max_depth = 2 min_samples_split = 2 Validation Accuracy = 0.9176470588235294
max_depth = 2 min_samples_split = 5 Validation Accuracy = 0.9176470588235294
max_depth = 2 min_samples_split = 10 Validation Accuracy = 0.9176470588235294
max_depth = 4 min_samples_split = 2 Validation Accuracy = 0.9647058823529412
max_depth = 4 min_samples_split = 5 Validation Accuracy = 0.9411764705882353
max_depth = 4 min_samples_split = 10 Validation Accuracy = 0.9411764705882353
max_depth = 6 min_samples_split = 2 Validation Accuracy = 0.9647058823529412
max_depth = 6 min_samples_split = 5 Validation Accuracy = 0.9529411764705882
max_depth = 6 min_samples_split = 10 Validation Accuracy = 0.9529411764705882
max_depth = 8 min_samples_split = 2 Validation Accuracy = 0.9411764705882353
max_depth = 8 min_samples_split = 5 Validation Accuracy = 0.9411764705882353
max_depth = 8 min_samples_split = 10 Validation Accuracy = 0.9294117647058824
max_depth = None min_samples_split = 2 Validation Accuracy = 0.9411764705882353
max_depth = None min_samples_split = 5 Validation Accuracy = 0.9411764705882353
max_depth = None min_samples_split = 10 Validation Accuracy = 0.9294117647058824
```



The plot shows how validation accuracy differs throughout distinct combinations of tree depth and minimum samples required for splitting. It shows that the highest validation accuracy was obtained with the following configurations:

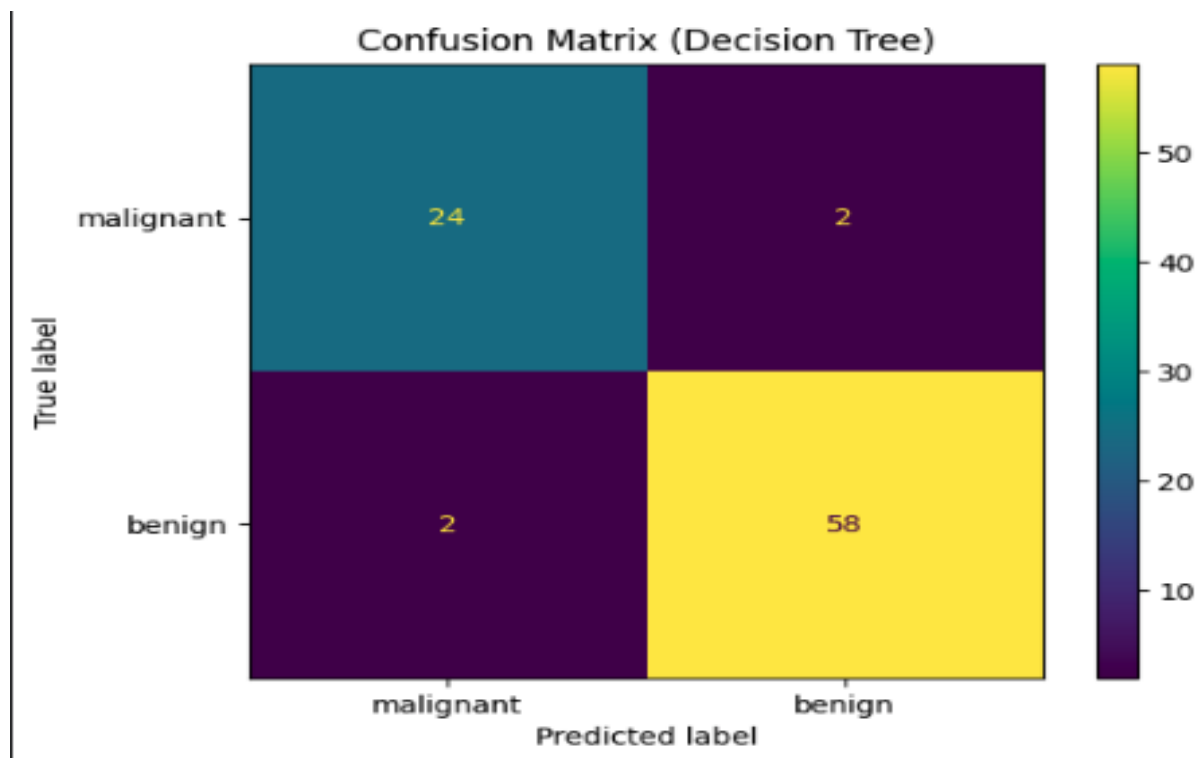
```
Best max_depth: 4
Best min_samples_split: 2
```

It was chosen based on the highest accuracy and the simplest tree possible. Consider that for the **max\_depth\_values = none** it was recorded as 10 in order to plot.

After selecting the best hyperparameters the decision tree was retrained using the combined training and validation datasets. Using more data during training helps the model learn representative patterns before evaluating on the unseen data.

The final model was then evaluated on the test dataset to measure its generalization performance. The final model achieved a 95.3% test accuracy with some misclassifications.

```
Decision Tree Test Accuracy: 0.9534883720930233
```

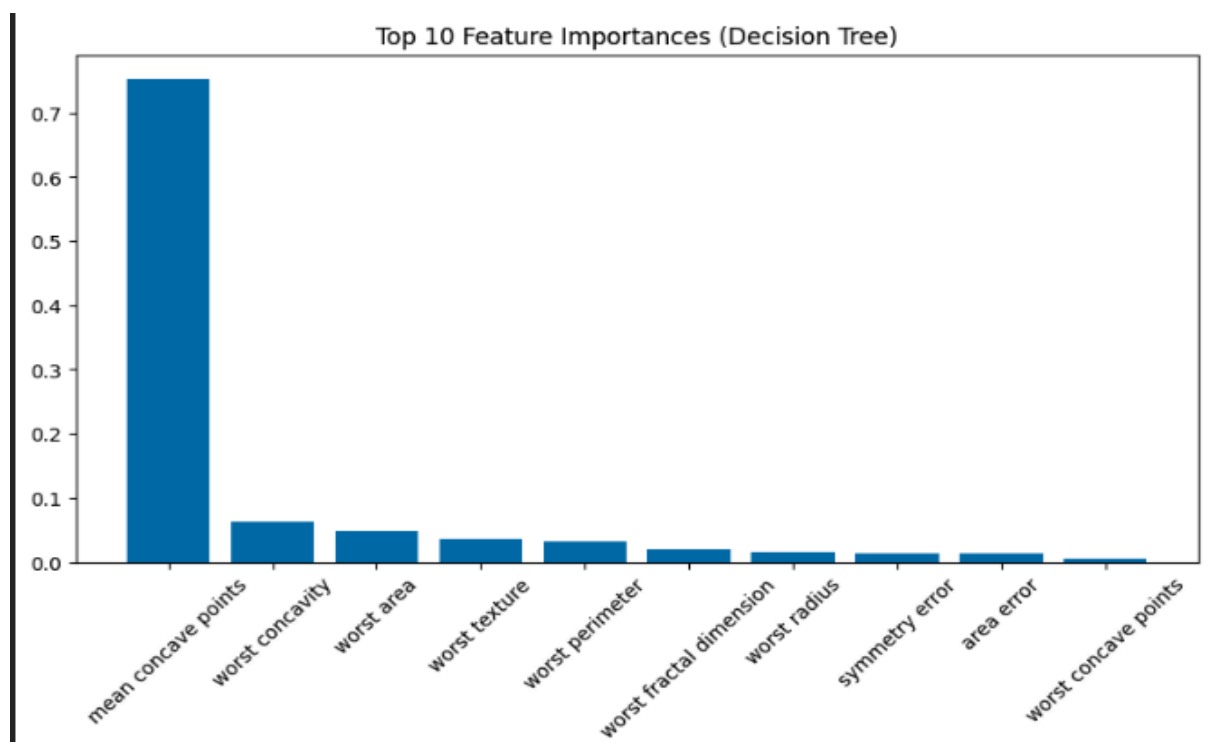


The **Confusion matrix** summarizes the classification results of the Decision Tree model.

**Confusion matrix Interpretation:**

- 24 malignant tumors were correctly classified as malignant (True Positives).
- 2 malignant tumors were incorrectly classified as benign (False Negatives).
- 58 benign tumors were correctly classified as benign (True Negatives).
- 2 benign tumors were incorrectly classified as malignant (False Positives)

Although the Decision Tree model achieves a high accuracy, it makes slightly more misclassifications compared to the k-NN model.



The feature importance plot shows which attributes contributed the most to the decision tree splits. The most informative feature identified by the model is mean concave points, followed by features such as worst concavity, worst area, and worst texture.

These features describe properties related to the shape and irregularity of tumor cells, which are known to be strong indicators of malignancy. Features with higher importance values have a greater influence on how the decision tree makes classification decisions.

### **3. Misclassification Analysis**

Even though the two models get high classification accuracy, there are a few misclassifications. Addressing these errors are vital, specifically in a medical context where those types of mistakes can cause harmful consequences.

For the k-NN classifier, the confusion matrix displays exactly a single malignant tumor that was misclassified as benign. This type of error is called a false negative, and it is extremely disapproving in medical diagnosis since it might make the treatment for a cancer patient further. Nonetheless, the model accurately classified all benign tumors and most malignant tumors, leading to an exceptionally powerful overall performance.

For the Decision Tree classifier, more mistakes were seen in comparison to the k-NN classifier. The model misclassified two malignant tumors as benign and two benign tumors as malignant. While false positives (benign predicted as malignant) may lead to unnecessary follow-up tests, they are generally less dangerous than false negatives.

Generally, both models perform well on this dataset, but the k-NN model produces fewer critical errors, making it slightly more reliable for this classification task.

### **4. Conclusion**

In this homework, two machine learning algorithms, k-Nearest Neighbors (k-NN) and Decision Tree, were used to classify tumors in the Breast Cancer Wisconsin dataset as malignant or benign. In which the dataset went through analysis and preprocessing, involving data splitting into subsets for training, validation, and test sets and feature standardization to ensure fair distance calculations for the k-NN algorithm.

The k-NN classifier achieved the best performance, reaching a test accuracy of approximately 98.8% with the optimal parameter  $k = 9$ . The Decision Tree classifier also performed well, but it produced slightly more misclassifications compared to the k-NN model with a 95.3%.

The confusion matrices showed that both models correctly classified the majority of tumors, with only a small number of errors. In the medical context, avoiding false negatives (predicting malignant tumors as benign) is particularly important, and the k-NN model produced fewer such errors.

Additionally, the Decision Tree model provided insight into which features were most informative for classification. The feature importance analysis revealed that characteristics related to tumor shape and irregularity, such as mean concave points and worst concavity, play a significant role in distinguishing between malignant and benign tumors.

Overall, both models demonstrate that machine learning methods can be highly effective for medical diagnosis tasks, and the results highlight the importance of proper preprocessing, hyperparameter tuning, and model evaluation when building reliable classification systems.